
Digital Forensics

Topic 5: Anti-Forensics & Cryptography

Lab 5: File Structure & Data Recovery

F20FO/F21FO – Exam-Ready Master Notes

Heriot-Watt University – 2025/26

Contents

1	Introduction to Anti-Forensics	2
1.1	The Big Picture (Why This Matters)	2
1.2	Incidental Anti-Forensics Techniques	2
1.3	Intentional Anti-Forensics Techniques	2
2	Steganography	4
2.1	Steganography vs Cryptography — The Critical Difference	4
2.2	How Steganography Works (The Process)	4
2.3	Historical Examples of Steganography	4
2.4	Modern Steganography Techniques	5
2.4.1	1. Masking and Filtering	5
2.4.2	2. Algorithms and Transformations	5
2.4.3	3. Least Significant Bit (LSB) Insertion — THE MOST IMPORTANT	5
2.5	Types of Steganography by Media	6
2.6	Steganalysis — Detecting Steganography	6
3	Cryptography	8
3.1	Essential Cryptography Terminology	8
3.2	What Cryptography Provides (The Four Pillars)	8
3.3	Where Cryptography Is Used	9
3.4	Cryptography Techniques: Transposition vs Substitution	10
3.4.1	Transposition Ciphers	10
3.4.2	Substitution Ciphers	10
3.5	Caesar Cipher — Full Breakdown	11
3.5.1	The Caesar Cipher Alphabet Table	11
3.5.2	Why Caesar Cipher Is Weak	11
3.6	ROT13 — A Special Caesar Cipher	12
3.7	Vigenère Cipher	12
3.7.1	Breaking the Vigenère Cipher	13

3.8	Breaking Substitution Ciphers — Frequency Analysis	14
3.8.1	How to Detect Transposition vs Substitution	14
3.9	Key Space — Why Size Matters	14
4	Symmetric vs Asymmetric Encryption	15
4.1	Symmetric Encryption	15
4.1.1	Types of Symmetric Encryption	15
4.1.2	The Key Distribution Problem	15
4.2	Asymmetric (Public Key) Encryption	15
4.3	Symmetric vs Asymmetric — Complete Comparison	16
5	Digital Signatures and Certificates	17
5.1	Digital Signatures	17
5.2	Certificates and Trust	17
5.2.1	The Trust Problem	17
5.2.2	How HTTPS Works (Using Certificates)	18
5.2.3	Weaknesses of Certificate Authorities	18
6	Cryptanalysis Attack Types	19
7	Implications for Digital Forensics	20
7.1	Step 1: Detect Whether Anti-Forensics Was Used	20
7.2	Step 2: If Encrypted Content Is Found	20
7.3	File Signatures for Encrypted Files	20
7.4	Using Entropy to Detect Encrypted Files	21
8	Lab 5: File Structure and Data Recovery	22
8.1	The 5 Components of a File	22
8.2	File Signatures (Magic Numbers) — Critical for Exams	22
8.3	Common File Signatures Table	23
8.4	JPEG Signature — Detailed Breakdown	23
8.5	File Carving — Recovering Deleted Files	24
8.5.1	Why File Carving Works	24
8.5.2	Manual File Carving — Step by Step	24
8.5.3	Files Without Footers	24
8.6	Automated File Carving with Scalpel	25
8.6.1	How Scalpel Works	25
8.6.2	Using Scalpel — Commands	25
9	Number Systems Review (From Lab 5)	26
9.1	Quick Conversion Reference	26
9.2	Solving the Lab Conversion Problems	26
9.3	RGB Colour Codes	27
10	Exam Quick Reference / Cheat Sheet	28
10.1	Anti-Forensics at a Glance	28
10.2	Steganography vs Cryptography	28
10.3	Cryptography Quick Facts	28
10.4	File Forensics Quick Facts	28
10.5	Attack Types	28

1 Introduction to Anti-Forensics

DEFINITION

Anti-Forensics is any technique, tool, or method used—either **intentionally or accidentally**—that makes it **harder** for a digital forensics (DF) expert to find, recover, or analyse digital evidence.

1.1 The Big Picture (Why This Matters)

Think of it this way: in previous topics, you learned how DF experts *find* evidence (file carving, file signatures, metadata, etc.). Anti-forensics is the **other side of the coin**—it’s everything that can **block or hinder** that process.

There are two categories of anti-forensics:

REAL-WORLD ANALOGY

Imagine you’re a detective searching a house for evidence.

- **Incidental anti-forensics** = The owner had a shredder for privacy and happened to shred some receipts before you arrived. They weren’t trying to hide from *you specifically*.
- **Intentional anti-forensics** = The owner *knew* the police were coming, so they burned documents, poured bleach on evidence, and hid things in the walls.

1.2 Incidental Anti-Forensics Techniques

These are things people do for **normal, legitimate reasons** that *happen* to make forensics harder. The user did NOT intend to interfere with any investigation.

Technique	What It Is (ELI5)	Why It Hurts Forensics
Wear Levelling	Flash drives (SSDs, USBs) spread data evenly across all cells so no single cell wears out first. Like rotating tyres on a car.	Data gets moved around automatically, so “deleted” data may be overwritten without the user even knowing.
Encryption	Scrambling your data so only you can read it (e.g., BitLocker, FileVault). Like putting your diary in a locked safe.	Even if the DF expert finds the file, they can’t read it without the key/password.
Defragmentation	The OS reorganises scattered file pieces to be together on disk. Like tidying a messy bookshelf.	Moves data around, which can overwrite unallocated space that contained old deleted files.
Advanced Deletion	Software that truly erases data by writing over it multiple times (e.g., secure delete tools).	The original data is gone for good —file carving won’t work.

1.3 Intentional Anti-Forensics Techniques

A criminal who knows they might be investigated may proactively:

1. **Encrypt** their storage data (so investigators can't read files)
2. **Periodically defragment** their drives (to overwrite old evidence in unallocated space)
3. **Permanently delete** data by **overwriting** it, not just “deleting” it from the file system

KEY EXAM POINT

Remember from earlier topics: when you “delete” a file normally, the OS just marks those bytes as “available for reuse.” The actual data is **still there** until something else writes over it. That's why file carving works. Intentional anti-forensics **overwrites** the actual bytes, making recovery impossible.

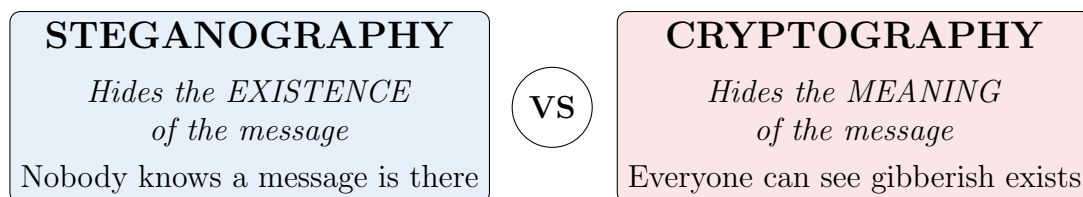
2 Steganography

DEFINITION

Steganography (from Greek: “hidden writing”) is the technique of **hiding a secret message inside a normal-looking file** (image, audio, video) so that nobody even *suspects* a hidden message exists.

2.1 Steganography vs Cryptography — The Critical Difference

This is a **very common exam question**. Make sure you understand the difference:



REAL-WORLD ANALOGY

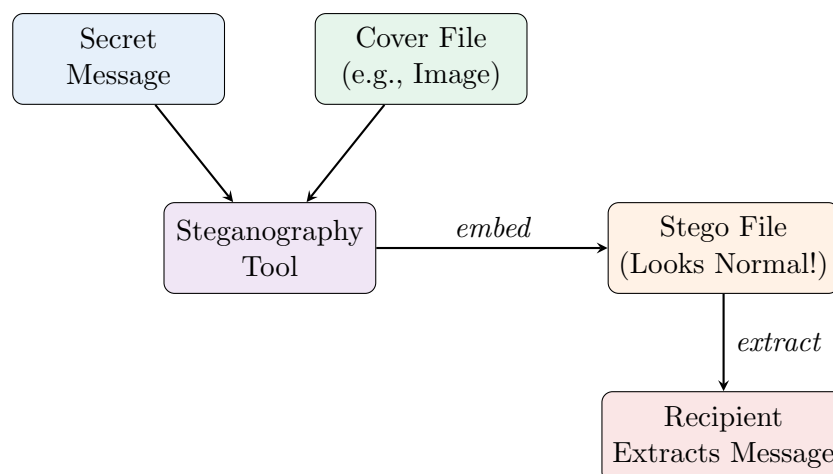
Steganography = Writing a secret message in invisible ink on a normal letter. The postman sees a regular letter.

Cryptography = Writing a message in a code language. The postman sees the letter but can't understand it—and they *know* it's in code.

KEY EXAM POINT

Key exam point: Steganography hides the **existence** of the message. Cryptography hides the **meaning**. They can also be **combined**—you can encrypt a message AND THEN hide it inside an image.

2.2 How Steganography Works (The Process)



2.3 Historical Examples of Steganography

The exam may ask about the history. Here are the key stories:

Who / When	What They Did
Histiaeus (485–525 BC)	Shaved a messenger's head, tattooed a secret message on his scalp, waited for hair to regrow, then sent him. The recipient shaved the messenger's head to read the message.
Pliny the Elder	Used invisible ink made from plant milk. Dried clear on paper, but turned brown when heated.
Johannes Trithemius (1518)	Created a cipher where each letter became a word, and the words formed a legitimate-looking prayer.
Giovanni Porta (1560)	Wrote messages on a hard-boiled egg shell using a special ink (alum + vinegar). The ink penetrated the shell and appeared on the egg white inside.

2.4 Modern Steganography Techniques

There are **three main techniques** used in digital steganography:

2.4.1 1. Masking and Filtering

- Hides information inside an image using **digital watermarks**
- The watermark contains info like copyright, ownership, or license data
- **Different from traditional steganography** because the goal is to add metadata to the image, not to secretly communicate
- Think of it like a stamp on a photo that proves who owns it

2.4.2 2. Algorithms and Transformations

- Hides data inside mathematical functions used in **compression algorithms**
- The secret message is hidden in the **least significant coefficients** of the compression math
- More complex, harder to detect

2.4.3 3. Least Significant Bit (LSB) Insertion — THE MOST IMPORTANT

MUST MEMORIZE

LSB Insertion is the most common and most popular method. It is the one you are most likely to be asked about in the exam.

Here's how it works, explained step by step:

WORKED EXAMPLE**How LSB Insertion Works:**

Every pixel in an image is stored as numbers. For example, a pixel might be stored as 8 bits (binary digits) for its red value:

Original pixel red value: 10110100

The **Least Significant Bit (LSB)** is the **last bit on the right**. It's called "least significant" because changing it barely changes the colour.

1011010 **0** ← *This is the LSB*

To hide a secret bit of "1", we change the LSB:

1011010 **1** ← *Changed from 0 to 1*

Original value: $10110100_2 = 180$ (decimal)

Modified value: $10110101_2 = 181$ (decimal)

The colour changed by just **1 out of 255**. **The human eye cannot tell the difference!**

By changing the LSB of thousands of pixels, you can hide an entire message inside an image that looks completely identical to the original.

KEY EXAM POINT

LSB works best when: (1) the image file is **larger than** the message file, and (2) the image is **grayscale** (fewer colour channels = less visible distortion).

2.5 Types of Steganography by Media

Type	File For- mats	Techniques Used	Example Tools
Image	PNG, JPEG, BMP	Masking/Filtering, Algorithms, LSB Insertion	iSteg, Steghide, OutGuess, EzStego
Audio	MP3, WAV	Algorithms, LSB Insertion	WavSteg (Python-based, command line)
Video	AVI, MP4, WMV	Both image & audio techniques (since video = images + audio)	MSU Stego Video

2.6 Steganalysis — Detecting Steganography**DEFINITION**

Steganalysis is the detection of steganography. It's the process of figuring out whether a file contains a hidden message.

There are **two main methods**:

1. **Visual Analysis:** Try to see the hidden data with your eyes or with computer assistance. For example, zooming into an image and looking for unusual pixel patterns.

2. **Statistical (Algorithmic) Analysis:** Analyse the file's statistical properties. Steganographic embedding causes tiny changes in the file's normal statistical behaviour that software can detect.

KEY EXAM POINT

Important distinction for the exam:

- **Universal** steganalysis methods (designed to detect *any* hidden data) are generally **less effective**
- **Specific** steganalysis methods (designed to detect a *particular* embedding technique) are **more effective**

Tool example: **Stegdetect** can detect the use of steganography in files.

3 Cryptography

DEFINITION

Cryptography (from Greek: “secret writing”) is the science of hiding the **meaning** of a message by transforming it into an unreadable format. Only someone with the correct **key** can transform it back.

3.1 Essential Cryptography Terminology

MUST MEMORIZE

You **MUST** know every single term in this table. They appear in almost every exam.

Term	Meaning (ELI5)	Example
Plaintext	The original, readable message	“HELLO”
Ciphertext	The scrambled, unreadable output	“KHOOR”
Cipher	The algorithm/recipe used to scramble	Caesar Cipher
Key	The secret piece of info that controls how the cipher works	The number 3
Encipher (Encrypt)	Converting plaintext → ciphertext	“HELLO” → “KHOOR”
Decipher (Decrypt)	Converting ciphertext → plaintext	“KHOOR” → “HELLO”
Cryptography	The <i>study</i> of encryption methods	This whole section
Cryptanalysis	The <i>study</i> of breaking encryption without the key	Code-breaking
Cryptology	Both cryptography AND cryptanalysis together	The whole field

3.2 What Cryptography Provides (The Four Pillars)

Cryptography can provide four key security properties:

Confidentiality

Only intended recipient reads it

Integrity

Message hasn't been changed

Authentication

Proves who sent it

Non-repudiation

Sender can't deny they sent it

REAL-WORLD ANALOGY

Think of sending a sealed, signed letter via registered post:

- **Confidentiality** = The envelope is sealed (only the recipient opens it)
- **Integrity** = If someone opened the envelope, you'd see the broken seal
- **Authentication** = Your signature proves it's from you
- **Non-repudiation** = The post office has a record that YOU sent it—you can't deny it later

3.3 Where Cryptography Is Used

Cryptography is everywhere in modern life:

- **VPNs** (Virtual Private Networks) — encrypts your internet connection
- **Password storage** — stores hashes, not actual passwords
- **Internet banking** — HTTPS encrypts your bank transactions
- **Messaging apps** — WhatsApp, Telegram use end-to-end encryption
- **Wi-Fi** — WPA2/WPA3 encrypts wireless data
- **Full disk encryption** — BitLocker, FileVault

3.4 Cryptography Techniques: Transposition vs Substitution

There are two fundamental approaches to encrypting data. The exam *loves* asking about these.

3.4.1 Transposition Ciphers

DEFINITION

A **transposition cipher** rearranges the characters of the plaintext without changing them. The same letters appear, just in a different order.

WORKED EXAMPLE

Columnar Transposition — Step by Step:

Plaintext: HELLO THERE

Step 1: Choose a number of columns (say 4). Write the message in rows of 4:

Col 1	Col 2	Col 3	Col 4
H	E	L	L
O		T	H
E	R	E	

Step 2: Read down each column from left to right:

Column 1: H, O, E → HOE

Column 2: E, , R → E R

Column 3: L, T, E → LTE

Column 4: L, H → LH

Ciphertext: HOE E R LTE LH

Notice: **Every original letter is still there!** They're just rearranged.

KEY EXAM POINT

How to detect a transposition cipher:

The **single letter frequencies** in the ciphertext are the **same** as in English plaintext (because no letters were changed, just moved). But the **double/triple letter frequencies** will be different (because pairs like “TH” got broken up).

3.4.2 Substitution Ciphers

DEFINITION

A **substitution cipher** replaces each character with a different character. The positions stay the same, but the letters change.

Feature	Transposition	Substitution
What changes?	Position of letters	Identity of letters
Letters the same?	Yes, same letters	No, different letters
Single letter freq.?	Same as plaintext	Different from plaintext
Example	Columnar transposition	Caesar cipher

3.5 Caesar Cipher — Full Breakdown

DEFINITION

The **Caesar Cipher** is the simplest substitution cipher. It shifts every letter in the alphabet by a fixed number of positions (the **key**).

3.5.1 The Caesar Cipher Alphabet Table

With a key of 3 (Julius Caesar’s original choice):

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Top row = plaintext letter, Bottom row = ciphertext letter.

WORKED EXAMPLE

Encrypt “HERIOT” with Caesar Cipher, Key = 3

Go letter by letter and shift each one forward by 3 in the alphabet:

	Letter 1	Letter 2	Letter 3	Letter 4	Letter 5	Letter 6
Plaintext	H	E	R	I	O	T
Shift +3	H→I→J→K	E→F→G→H	R→S→T→U	I→J→K→L	O→P→Q→R	T→U→V→W
Ciphertext	K	H	U	L	R	W

Result: HERIOT → KHULRW

To **decrypt**, just shift **backwards** by 3: K→J→I→H, and so on.

WORKED EXAMPLE

What about letters near the end? (Wrap-around)

Encrypt “FUZZY” with Key = 5:

- $F + 5 = K$
- $U + 5 = Z$
- $Z + 5 = Z \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$ (wraps around!)
- $Z + 5 = E$
- $Y + 5 = Y \rightarrow Z \rightarrow A \rightarrow B \rightarrow C \rightarrow D$

Result: FUZZY → KZEED

The alphabet **wraps around** from Z back to A.

3.5.2 Why Caesar Cipher Is Weak

COMMON EXAM MISTAKE

Caesar Cipher has only **26 possible keys** (or 27 if you include spaces). This is an **extremely small key space**. An attacker only needs to try on average **13 keys** to crack it. If each attempt takes 1 minute, it takes approximately **13 minutes** to brute-force. This is essentially useless for real security.

KEY EXAM POINT

Caesar Cipher relies on “**security through obscurity**”—it only works if the attacker doesn’t know the method. Once they know it’s a Caesar Cipher, it’s trivially broken.

3.6 ROT13 — A Special Caesar Cipher**DEFINITION**

ROT13 is a Caesar Cipher with a key of exactly 13. Since 13 is half of 26 (the alphabet size), applying ROT13 **twice** returns you to the original text.

A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M

Key property: **Encrypt = Decrypt**. Applying ROT13 once encrypts. Applying it again decrypts.

ROT13 is **NOT designed for real security**. It’s used on forums to hide spoilers, quiz answers, etc. However, it was mistakenly used in Netscape Communicator to store email passwords—a serious security flaw.

3.7 Vigenère Cipher**DEFINITION**

The **Vigenère Cipher** is a more advanced substitution cipher that uses a **keyword** instead of a single number. Each letter of the keyword tells you how much to shift the corresponding plaintext letter.

WORKED EXAMPLE**Encrypt “DIGITAL” with keyword “CAT”**

Step 1: Repeat the keyword over the plaintext:

Plaintext	D	I	G	I	T	A	L
Keyword	C	A	T	C	A	T	C

Step 2: Convert each keyword letter to a shift number (A=0, B=1, C=2, ..., T=19, ...):

$$C = 2, \quad A = 0, \quad T = 19$$

Step 3: Shift each plaintext letter by the corresponding keyword number:

Plaintext	D	I	G	I	T	A	L
Shift by	+2	+0	+19	+2	+0	+19	+2
Ciphertext	F	I	Z	K	T	T	N

Let's verify each:

- $D + 2 = F$ ✓
- $I + 0 = I$ ✓
- $G + 19 = G(7) + 19 = 26 \rightarrow$ wraps to Z(25... actually: $G=6, 6+19=25=Z$) ✓
- $I + 2 = K$ ✓
- $T + 0 = T$ ✓
- $A + 19 = T$ ✓
- $L + 2 = N$ ✓

Result: DIGITAL $\rightarrow$ FIZKTTN

3.7.1 Breaking the Vigenère Cipher

Because the keyword **repeats**, if you can figure out the keyword length, you can break it:

1. Determine the keyword length (e.g., 3 for “CAT”)
2. Look at every 3rd letter—they're all shifted by the same amount
3. Use **frequency analysis** on each group to figure out which letter is “E” (the most common English letter)
4. Once you know the shift for each position, you know the keyword

3.8 Breaking Substitution Ciphers — Frequency Analysis

MUST MEMORIZE

English letter frequencies (most common first):

E: 12.7% **T:** 9.1% **A:** 8.1%

Most common letter pairs: TH, IN, HE, AN, RE, ER

If the exam gives you ciphertext, count which letter appears most. That's probably **E** encrypted.

3.8.1 How to Detect Transposition vs Substitution

What to Check	Transposition	Substitution (e.g., Caesar)
Single letter frequencies	Same as English	Shifted (same shape but different letters)
Double/triple letter freq.	Different from English	Shifted version of English pairs
Detection method	Single freqs match → transposition	Graph shape matches but letters shifted → substitution

3.9 Key Space — Why Size Matters

DEFINITION

The **key space** is the total number of possible keys an encryption algorithm can use. A bigger key space = harder to crack by brute force.

Algorithm	Key Space	Security Level
Caesar Cipher	26 keys	Completely useless
56-bit key	$2^{56} = 72$ quadrillion keys	Crackable in ~24 hours by supercomputer
64-bit key	2^{64} keys	Marginal
128-bit key	2^{128} keys	Practically uncrackable
256-bit key	2^{256} keys	Military grade

KEY EXAM POINT

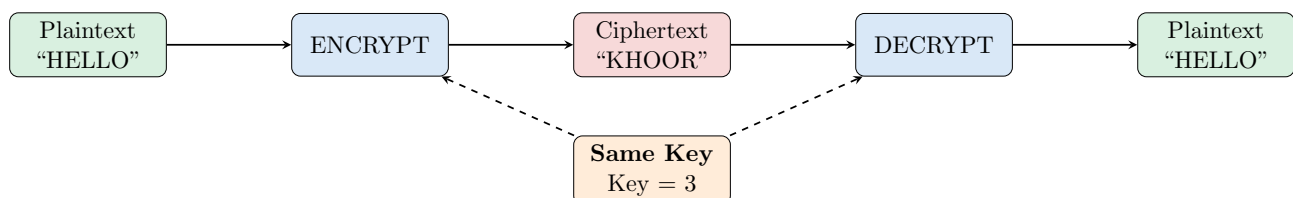
Critical fact: Each additional bit **doubles** the key space. So a 128-bit key is not “a bit better” than a 56-bit key—it's 2^{72} times harder to crack. That's approximately **4.7 sextillion times** harder. A supercomputer that cracks 56-bit in 24 hours would need **trillions of years** for 128-bit.

4 Symmetric vs Asymmetric Encryption

4.1 Symmetric Encryption

DEFINITION

Symmetric encryption uses the **same key** for both encryption and decryption. Both sender and receiver must know the same secret key.



4.1.1 Types of Symmetric Encryption

	Block Cipher	Stream Cipher
How it works	Encrypts data in fixed-size blocks (e.g., 64 or 128 bits at a time)	Encrypts data one bit or byte at a time
Best for	Single messages, files	Continuous data streams (e.g., live video)
Analogy	Packing boxes: fill one box, seal it, start the next	A conveyor belt: items go through one at a time
Example	AES, DES	RC4

4.1.2 The Key Distribution Problem

COMMON EXAM MISTAKE

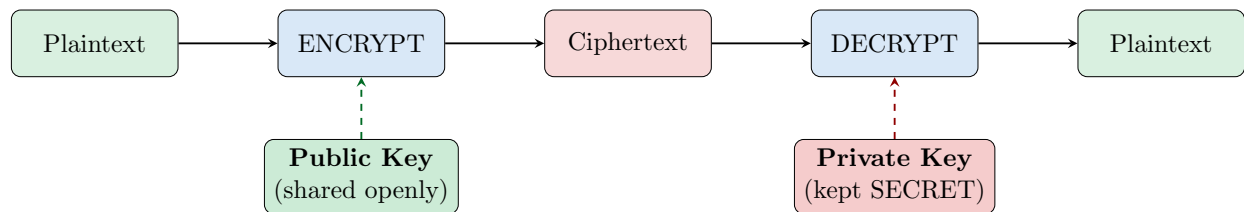
The biggest weakness of symmetric encryption: **How do you securely share the key?** If Alice wants to talk secretly with Bob, they need the same key. But how does Alice send the key to Bob without anyone intercepting it? This is the **Key Distribution Problem**.

If Alice also talks to Carol and Dave, she needs a **separate key for each person**. With n people, you need $\frac{n(n-1)}{2}$ keys total—this gets unmanageable fast!

4.2 Asymmetric (Public Key) Encryption

DEFINITION

Asymmetric encryption uses **two different keys**: a **public key** (for encryption) and a **private key** (for decryption). Anyone can encrypt with the public key, but only the private key holder can decrypt.



REAL-WORLD ANALOGY

Think of a **padlock and key**:

- Bob gives everyone an **open padlock** (public key). Anyone can lock a box with it.
- Only Bob has the **key** to open the padlock (private key).
- Alice puts her message in a box, locks it with Bob's padlock, and sends it.
- Only Bob can open it, even though anyone could have locked it!

4.3 Symmetric vs Asymmetric — Complete Comparison

	Symmetric	Asymmetric
Keys used	ONE key (same for encrypt & decrypt)	TWO keys (public + private)
Speed	Very fast	Up to 10,000x slower
Key sharing	Must be shared secretly (problem!)	Public key can be shared openly
Key management	Need $\frac{n(n-1)}{2}$ keys for n people	Each person needs only 1 key pair
Examples	AES, DES, Caesar Cipher	RSA, Diffie-Hellman
Used for	Encrypting large amounts of data	Key exchange, digital signatures

KEY EXAM POINT

In practice, **both are used together**: asymmetric encryption is used to securely exchange a symmetric key, then symmetric encryption is used for the actual data (because it's much faster). This is how HTTPS works!

5 Digital Signatures and Certificates

5.1 Digital Signatures

DEFINITION

A **digital signature** is an application of public key cryptography that proves three things about a message:

1. **Authentication:** It was created by a known sender
2. **Integrity:** It hasn't been altered
3. **Non-repudiation:** The sender can't later deny signing it

How it works: The sender encrypts a message hash with their **private key**. Anyone can verify it using the sender's **public key**. If verification succeeds, the message is authentic and unaltered.

Uses: signing software, legal digital documents, code signing.

5.2 Certificates and Trust

DEFINITION

A **digital certificate** binds a public key to an identity (a person, company, or website). A trusted third party **digitally signs** the certificate to confirm the identity is real.

5.2.1 The Trust Problem

How do you know someone online is who they claim to be? Two systems exist to solve this:

1. Web of Trust:



Alice signs Bob's key. Carol trusts Alice, so Carol trusts Bob. Dave trusts Carol, so Dave can also trust Bob (through the chain).

2. Certificate Authorities (CAs):

- Centralised trusted organisations that issue certificates
- They verify identity (typically by checking the applicant can receive email at the claimed domain — called **domain validation**)
- CAs can be **hierarchical**: some CAs can create other CAs, forming a chain
- Your web browser comes pre-loaded with a list of trusted CAs

5.2.2 How HTTPS Works (Using Certificates)

WORKED EXAMPLE

When you visit `https://mybank.com`:

1. Your browser receives the website's **certificate**
2. Browser checks: is this certificate signed by a **trusted CA**? If yes, continue
3. Browser uses the **public key** from the certificate to encrypt communication
4. Only the bank's server (with the **private key**) can decrypt
5. An attacker trying a man-in-the-middle attack couldn't provide a valid CA-signed certificate

5.2.3 Weaknesses of Certificate Authorities

COMMON EXAM MISTAKE

CAs are not perfect:

- Some CAs issue certificates to almost anyone without proper checks
- Government-owned CAs could conduct man-in-the-middle attacks
- In 2001, VeriSign issued two Microsoft certificates to an unrelated imposter
- Comodo and DigiNotar have been compromised by attackers

6 Cryptanalysis Attack Types

MUST MEMORIZE

The exam frequently tests these three attack types. Know them cold:

Attack Type	What the Attacker Has	Goal
Ciphertext Only	ONLY the encrypted message. Nothing else.	Find the plaintext and/or the key
Known Plaintext	Both the original message AND its encrypted version	Figure out the key (so they can decrypt future messages)
Chosen Plaintext	The attacker gets to CHOOSE what message to encrypt, and sees the result	Figure out the key by analysing patterns

REAL-WORLD ANALOGY

- **Ciphertext Only** = You intercept a sealed letter but have no idea what's inside
- **Known Plaintext** = You find a draft of a letter AND its sealed version—now you can figure out how the seal works
- **Chosen Plaintext** = You trick someone into sealing a message YOU wrote—so you know exactly what went in and can study what came out

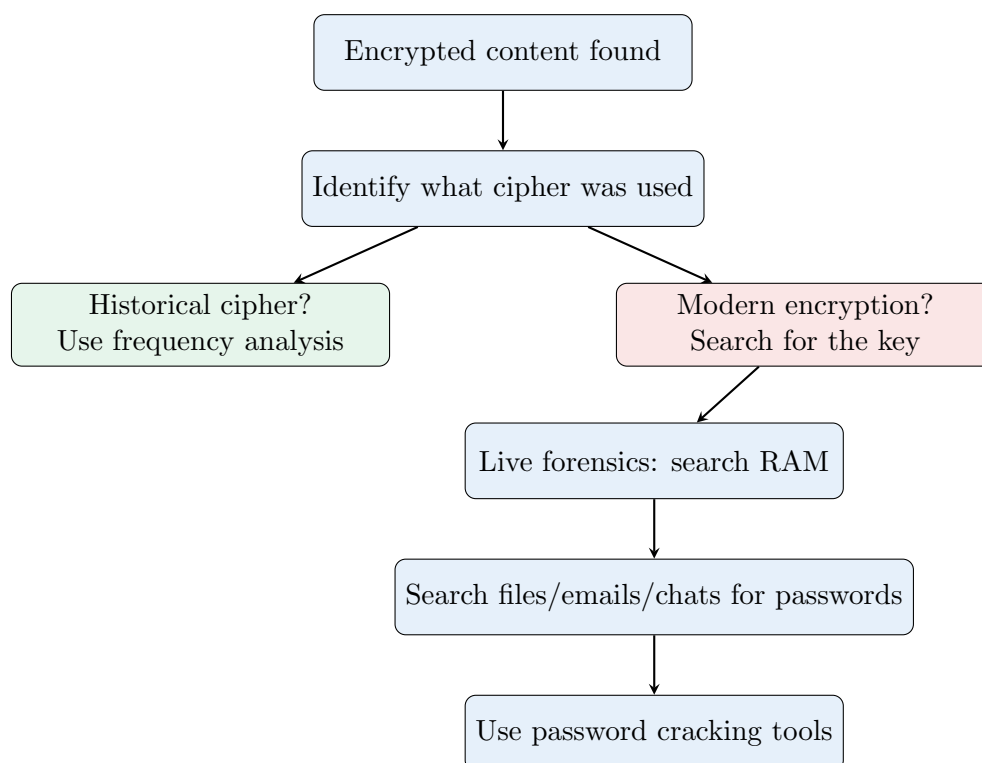
7 Implications for Digital Forensics

This section connects **everything** back to the forensics exam. How does a DF expert deal with anti-forensics?

7.1 Step 1: Detect Whether Anti-Forensics Was Used

Detection Method	How It Works
File Signature Analysis	Don't trust file extensions! Examine the actual hex header bytes to identify the true file type (see Lab section below)
Entropy Analysis	Calculate the randomness of a file's bytes. High entropy ($\approx 6-8$) suggests encryption or compression
Software Detection	Search the suspect's computer for steganography tools (iSteg, Steghide) or encryption programs (VeraCrypt, GPG)

7.2 Step 2: If Encrypted Content Is Found



7.3 File Signatures for Encrypted Files

Encrypted files have their own file signatures, just like any other file type:

WORKED EXAMPLE**Detecting OpenSSL Encrypted Files:**

A normal text file has no special signature—the OS detects it as ASCII text.

If you encrypt it with OpenSSL (AES-256), the encrypted file gets a signature:

53 61 6c 74 65 64 5f 5f

Converting this hex to ASCII:

53=S, 61=a, 6c=l, 74=t, 65=e, 64=d, 5f=_, 5f=_

So the file starts with the text ‘‘Salted_’’ — this is the file signature for OpenSSL encrypted files!

For Base64-encoded encryption: The signature is different:

55 32 46 73 64 47 56 6b 58 31 = ‘‘U2FsdGVkX1’’

A DF examiner can search for these signatures to find encrypted files.

7.4 Using Entropy to Detect Encrypted Files**MUST MEMORIZE**

Entropy values (for byte-level data, range 0 to 8):

File Type	Typical Entropy	What This Means
English text (plaintext)	≈ 4.49	Low randomness, lots of patterns
Encrypted (binary/default)	≈ 6.05	High randomness (looks random)
Encrypted (Base64 encoding)	≈ 5.53	Medium-high (fewer possible bytes)
Truly random data	≈ 8.00	Maximum randomness

KEY EXAM POINT

Key exam insight: The encoding of the encrypted file affects its entropy. Binary-encoded ciphertext has **higher entropy** than Base64-encoded ciphertext (because Base64 uses only 64 characters, while binary uses all 256 possible byte values).

So detecting encrypted data via entropy is useful but **not always definitive**—different encodings can make encrypted data look less random.

A DF examiner should use **BOTH** file signatures AND entropy together for best results.

8 Lab 5: File Structure and Data Recovery

This lab section covers the **practical skills** tested in the exam: understanding how files are stored, how to identify them, and how to recover deleted data.

8.1 The 5 Components of a File

MUST MEMORIZE

Every file on a computer has up to **5 parts**:

#	Component	What It Is (ELI5)	Visible Where?
1	Filename	The name you see in File Explorer (e.g., photo)	File Explorer
2	Extension	The type indicator after the dot (e.g., .jpg)	File Explorer (if not hidden)
3	Content/Data	The actual data—the image pixels, the document text, etc.	Depends on file type
4	Header / Signature	The hex bytes at the start of the file that identify its true type	Only visible in a hex editor
5	Footer	The hex bytes at the end of the file (not all files have one)	Only visible in a hex editor

8.2 File Signatures (Magic Numbers) — Critical for Exams

DEFINITION

A **file signature** (also called **magic number** or **magic bytes**) is a sequence of hex bytes at the very beginning of a file that identifies what type of file it truly is—**regardless of what the file extension says**.

KEY EXAM POINT

Why this matters for forensics:

Windows identifies files by their **extension** (.jpg, .pdf, etc.). This means a criminal can rename `IllegalItem.jpg` to `lansys.dll` and hide it in a system folder—Windows won't know the difference!

Linux identifies files by their **header/signature**. So on Linux, changing the extension won't fool the OS—but if you *corrupt the header*, the file won't open at all.

A forensics examiner should **ALWAYS check the file signature**, never trust the extension!

8.3 Common File Signatures Table

MUST MEMORIZE

File Type	Header (Hex)	Header (ASCII)	Footer (Hex)	
JPEG (.jpg)	FF D8 FF E0	(non-printable)	FF D9	
PNG (.png)	89 50 4E 47	.PNG	49 45 4E 44 AE 42 60 82	
PDF (.pdf)	25 50 44 46	%PDF	25 25 45 4F 46 (%%EOF)	
ZIP (.zip)	50 4B 03 04	PK..	3C (varies)	
WAV (.wav)	52 49 46 46	RIFF	No footer	
AVI (.avi)	52 49 46 46	RIFF???AVI	(varies)	
GIF (.gif)	47 49 46 38	GIF8	00 3B	

8.4 JPEG Signature — Detailed Breakdown

Byte	1	2	3	4	5	6	7	8	9	10	11
Hex	FF	D8	FF	E0	–	–	4A	46	49	46	00
ASCII							J	F	I	F	

- **FF D8**: Start of Image (SOI) marker — identifies it as a JPEG
- **FF E0**: Application marker for **JFIF** format (JPEG File Interchange Format)
- **FF E1**: Would indicate **EXIF** format instead (stores extra info like GPS location, camera model)
- **4A 46 49 46**: Spells “JFIF” in ASCII
- **Footer**: FF D9 = End of Image (EOI)

KEY EXAM POINT

If a JPEG uses FF E1 instead of FF E0, it uses **EXIF** (Exchangeable Image Format). EXIF stores metadata like the camera model, date taken, and GPS coordinates. This is forensically very valuable! You can tell what camera took the photo and where.

8.5 File Carving — Recovering Deleted Files

DEFINITION

File carving is the process of recovering files from raw data (like unallocated disk space) by searching for file headers and footers in the hex data, then extracting everything between them.

REAL-WORLD ANALOGY

Imagine you have a pile of shredded documents mixed together. File carving is like looking for the letterhead (header) of a specific company at the top of a page and the signature (footer) at the bottom, then piecing together everything between them.

8.5.1 Why File Carving Works

Remember: when you “delete” a file, the OS just marks those disk sectors as “available.” The actual bytes (including the header, content, and footer) are **still there** until something overwrites them. File carving finds those bytes.

8.5.2 Manual File Carving — Step by Step

WORKED EXAMPLE

Carving a PNG from raw data using a hex editor (Frhed):

Step 1: Open the raw data file in a hex editor

Step 2: Search for the PNG header:

89 50 4E 47 (in Frhed: <bh:89><bh:50><bh:4e><bh:47>)

Step 3: Note the byte offset where the header starts

Step 4: Search for the PNG footer:

49 45 4E 44 AE 42 60 82

Step 5: Select everything from header to footer (inclusive)

Step 6: Copy → New File → Paste (use INSERT mode, not overwrite!) → Save as **recovered.png**

Step 7: Open the file — you’ve recovered the image!

8.5.3 Files Without Footers

COMMON EXAM MISTAKE

Some file formats (like **WAV**) have **no footer**. This makes carving harder because you don’t know where the file ends. Solutions:

- Look for patterns in the hex data—large blank spaces (lots of 00 bytes) often indicate the end
- Look for the next file’s header (the next file starts where this one ends)
- Trial and error: copy different amounts of data and see what plays/opens correctly

8.6 Automated File Carving with Scalpel

For large datasets with many unknown files, manual carving is impractical. **Scalpel** automates the process.

DEFINITION

Scalpel is a free, open-source file carving tool that automatically searches raw data for file headers and footers, then extracts the files.

8.6.1 How Scalpel Works

1. It reads a **configuration file** (`scalpel.conf`) that lists file types with their headers and footers
2. Lines starting with **#** are comments (disabled). Remove **#** to enable a file type
3. It scans the raw data for all enabled headers/footers
4. It extracts matching files into a `scalpel-output` directory

8.6.2 Using Scalpel — Commands

```
1 # 1. Copy scalpel.conf from conf/ to root Scalpel/ folder
2 # 2. Edit scalpel.conf: remove # before jpg, zip, avi lines
3 #   (Use the jpg lines under "GRAPHICS FILES" section)
4 # 3. Run:
5 Scalpel.exe unknownChunk
6
7 # Output goes to scalpel-output/ directory
8 # Contains: folders for each file type + audit.txt
9
10 # NOTE: Delete scalpel-output folder before re-running!
```

Listing 1: Running Scalpel

KEY EXAM POINT

Key Scalpel facts for the exam:

- Scalpel uses a **configuration file** to know which file types to search for
- Configuration uses **#** for comments (like Python)
- Scalpel won't run if `scalpel-output` folder already exists — delete it first
- Use the jpg lines from the “GRAPHICS FILES” section, not the earlier example lines

9 Number Systems Review (From Lab 5)

9.1 Quick Conversion Reference

Decimal	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

9.2 Solving the Lab Conversion Problems

WORKED EXAMPLE

Convert binary 11001011 to decimal and hex:

Binary → Decimal:

$$1 \times 2^7 + 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\ = 128 + 64 + 0 + 0 + 8 + 0 + 2 + 1 = \mathbf{203}$$

Binary → Hex:

Split into groups of 4 from the right: 1100 1011

1100 = C, 1011 = B

Answer: CB

WORKED EXAMPLE

Convert hex 2AF7 to decimal and binary:

Hex → Decimal:

$$2 \times 16^3 + A(10) \times 16^2 + F(15) \times 16^1 + 7 \times 16^0 \\ = 2 \times 4096 + 10 \times 256 + 15 \times 16 + 7 \times 1 \\ = 8192 + 2560 + 240 + 7 = \mathbf{10999}$$

Hex → Binary:

Convert each hex digit to 4-bit binary:

2 = 0010, A = 1010, F = 1111, 7 = 0111

Answer: 0010 1010 1111 0111

WORKED EXAMPLE**Convert decimal 54 to binary and hex:****Decimal → Binary** (repeated division by 2):

$$54 \div 2 = 27 \text{ remainder } 0$$

$$27 \div 2 = 13 \text{ remainder } 1$$

$$13 \div 2 = 6 \text{ remainder } 1$$

$$6 \div 2 = 3 \text{ remainder } 0$$

$$3 \div 2 = 1 \text{ remainder } 1$$

$$1 \div 2 = 0 \text{ remainder } 1$$

Read remainders bottom to top: **110110****Decimal → Hex:**

$$54 \div 16 = 3 \text{ remainder } 6$$

$$3 \div 16 = 0 \text{ remainder } 3$$

Read bottom to top: **36****Verify:** $3 \times 16 + 6 = 48 + 6 = 54 \checkmark$

9.3 RGB Colour Codes

Each colour is defined as a mix of Red, Green, Blue values (0–255 each, or 00–FF in hex).

Colour	R (Hex)	G (Hex)	B (Hex)	R (Dec)	G (Dec)	B (Dec)
Red	FF	00	00	255	0	0
Green	00	FF	00	0	255	0
Blue	00	00	FF	0	0	255
White	FF	FF	FF	255	255	255

10 Exam Quick Reference / Cheat Sheet

10.1 Anti-Forensics at a Glance

Incidental: Wear levelling, encryption for privacy, defragmentation, secure delete
Intentional: Deliberate encryption, deliberate defrag, permanent data overwriting
Detection methods: File signature analysis, entropy analysis, searching for tools

10.2 Steganography vs Cryptography

Steganography: Hides the **EXISTENCE** of a message (nobody knows it's there)
Cryptography: Hides the **MEANING** of a message (everyone sees gibberish)
Most common steg technique: LSB Insertion (changes last bit of each pixel)
Steganalysis: Visual analysis or statistical/algorithmic analysis
Specific detectors & **universal detectors**

10.3 Cryptography Quick Facts

4 pillars: Confidentiality, Integrity, Authentication, Non-repudiation
Symmetric: Same key for encrypt/decrypt (fast, key distribution problem)
Asymmetric: Public key encrypts, private key decrypts (slow, solves key sharing)
In practice: Asymmetric exchanges symmetric key, symmetric encrypts data
Caesar Cipher: Shift by n , key space = 26, trivially broken
ROT13: Caesar with $n = 13$, apply twice = original, NOT for security
Vigenère: Keyword-based shifts, breakable via frequency analysis on key-length groups
Transposition: Rearranges letters (same frequencies), **Substitution:** replaces letters (shifted freq.)

10.4 File Forensics Quick Facts

5 file parts: Filename, Extension, Content, Header/Signature, Footer
Windows: Uses extensions → easy to fool by renaming
Linux: Uses headers → harder to fool, but corrupted header = can't open
File carving: Find header, find footer, extract everything between
Scalpel: Automated carving tool using `scalpel.conf` config file
Key signatures: JPEG = FF D8 FF E0, PNG = 89 50 4E 47, WAV = RIFF (no footer)
EXIF vs JFIF: FF E1 = EXIF (has metadata), FF E0 = JFIF (basic)
Encrypted file entropy: Plaintext ≈ 4.5 , Binary cipher ≈ 6.0 , Base64 cipher ≈ 5.5

10.5 Attack Types

Ciphertext only: Attacker has only ciphertext → find plaintext/key
Known plaintext: Attacker has plaintext + ciphertext → find key
Chosen plaintext: Attacker chooses plaintext, gets ciphertext → find key